Java-решения алгоритмов графов P9-P11

https://gemini.google.com/app/3f326d8d02c111e6

User prompt: С P9 до конца на java простым кодом, так же подготовь защиту для каждой задачи и теорию и объясни каждую строчку кода

Response: Ниже представлен полный разбор практических работ **P9 (Пути)**, **P10 (Кратчайшие пути)** и **P11 (Задача коммивояжера)** из методического пособия. Весь код написан на чистой Java в максимально простом процедурном стиле (с использованием базовых массивов и `ArrayList`), чтобы на защите было легко объяснить логику без углубления в сложные конструкции языка.

Каждая глава содержит:

1. **Теоретическую справку** по всем алгоритмам раздела.
2. **Единый скомпилированный Java-класс**, где каждая строка прокомментирована, а после кода дана подробная расшифровка ключевых блоков.
3. **Вопросы и ответы для защиты** (типичные вопросы преподавателей по сложности, крайним случаям и структурам данных).

Глава 9. Пути (P9)

1. Теория к P9

- **Простой путь** — это путь в графе, в котором ни одна вершина не повторяется. Поиск такого пути между вершинами u и v выполняется с помощью обхода в глубину (DFS). Чтобы записывать сам путь, используется массив (или стек), куда вершина добавляется при входе в рекурсию и удаляется при откате назад (**бэктрекинг**), если путь зашел в тупик.
- **Эйлеров цикл** — замкнутый путь, проходящий через **каждое ребро** графа ровно один раз.
  - *Критерий Эйлера (теорема):* Неориентированный граф имеет эйлеров цикл тогда и только тогда, когда: 1) степени всех его вершин чётны; 2) все вершины с ненулевой степенью принадлежат одной компоненте связности.
  - *Алгоритм поиска (Флёри / Иерхольцера на стеке):* Переходим по ребрам, удаляя их из копии матрицы смежности. Когда заходим в тупик (нет доступных ребер), записываем вершину в ответ и откатываемся назад.
- **Гамильтонов цикл** — замкнутый простой путь, который проходит через **каждую вершину** графа ровно один раз и возвращается в стартовую. В отличие от эйлерова цикла, простого критерия существования нет (задача **NP-полная**). Решается полным перебором с возвратом (Backtracking) за $O(V!)$.

2. Код программы для P9 (Задачи 1–6)

```
import java.util.*; // Импортируем утилиты для работы со списками и очередями

public class P9_Paths {

    // ================= ЗАДАЧА 1: Наличие простого пути (Матрица смежности) =================
    public static boolean hasSimplePath(int[][] matrix, int curr, int target, boolean[] visited) {
        if (curr == target) return true; // Если дошли до целевой вершины — путь найден
        visited[curr] = true; // Помечаем текущую вершину как посещенную
        for (int next = 0; next < matrix.length; next++) { // Перебираем все вершины графа
            if (matrix[curr][next] != 0 && !visited[next]) { // Если есть ребро и вершина еще не посещена
                if (hasSimplePath(matrix, next, target, visited)) { // Рекурсивно идем в соседа
                    return true; // Если путь нашелся глубже, возвращаем true наверх
                }
            }
        }
        return false; // Если обошли всех соседей и не дошли до цели — пути из этой ветки нет
    }

    // ================= ЗАДАЧА 2: Запоминание простого пути в массиве =================
    public static boolean findSimplePath(int[][] matrix, int curr, int target, boolean[] visited, int[] pa
        visited[curr] = true; // Помечаем вершину посещенной
        path[pathLen[0]] = curr; // Записываем текущую вершину в массив пути
        pathLen[0]++; // Увеличиваем счетчик длины пути на 1

        if (curr == target) return true; // Если достигли цели, выходим (путь уже лежит в массиве path)

        for (int next = 0; next < matrix.length; next++) { // Ищем соседей
            if (matrix[curr][next] != 0 && !visited[next]) { // Если сосед доступен и не посещен
                if (findSimplePath(matrix, next, target, visited, path, pathLen)) { // Идем вглубь
                    return true; // Успешно нашли путь
                }
            }
        }
```

```
        pathLen[0]--; // БЭКТРЕКИНГ (откат): удаляем вершину из пути, так как зашли в тупик
        return false; // Возвращаем false для перехода к следующей ветке
    }

    // ================ ЗАДАЧА 3: Проверка наличия Эйлерова цикла ================
    public static boolean hasEulerCycle(int[][] matrix) {
        int n = matrix.length; // Количество вершин в графе
        int startNode = -1; // Первая попавшаяся вершина с ребрами для проверки связности

        // 1. Проверяем четность степеней всех вершин
        for (int i = 0; i < n; i++) { // Проходим по каждой вершине
            int degree = 0; // Счетчик степени текущей вершины
            for (int j = 0; j < n; j++) { // Считаем количество инцидентных ребер
                if (matrix[i][j] != 0) degree++; // Увеличиваем степень при наличии ребра
            }
            if (degree % 2 != 0) return false; // Если степень нечетная — эйлерова цикла точно нет
            if (degree > 0 && startNode == -1) startNode = i; // Запоминаем вершину для старта DFS
        }

        if (startNode == -1) return false; // В графе вообще нет ребер

        // 2. Проверяем, что все вершины с ребрами находятся в одной компоненте связности
        boolean[] visited = new boolean[n]; // Массив посещенных вершин
        dfsConnectivity(matrix, startNode, visited); // Запускаем обычный обход в глубину

        for (int i = 0; i < n; i++) { // Проверяем все вершины после обхода
            for (int j = 0; j < n; j++) {
                // Если у вершины i есть ребро, но мы до нее не дошли в DFS — граф несвязный
                if (matrix[i][j] != 0 && !visited[i]) return false;
            }
        }
        return true; // Все условия выполнены: степени четные, граф связный
    }

    // Вспомогательный DFS для проверки связности в Задаче 3
    private static void dfsConnectivity(int[][] matrix, int curr, boolean[] visited) {
        visited[curr] = true; // Отмечаем вершину
        for (int next = 0; next < matrix.length; next++) { // Смотрим соседей
            if (matrix[curr][next] != 0 && !visited[next]) { // Если есть ребро и не были там
                dfsConnectivity(matrix, next, visited); // Идем рекурсивно дальше
            }
        }
    }

    // ================ ЗАДАЧА 4: Запоминание вершин Эйлерова цикла в массиве ================
    public static int[] getEulerCycle(int[][] matrix, int start) {
        if (!hasEulerCycle(matrix)) return new int[0]; // Если цикла нет, возвращаем пустой массив

        int n = matrix.length; // Число вершин
        int[][] tempGraph = new int[n][n]; // Создаем копию матрицы, чтобы удалять пройденные ребра
        int edgeCount = 0; // Считаем общее число ребер для размера итогового массива
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                tempGraph[i][j] = matrix[i][j]; // Копируем элемент матрицы
                if (matrix[i][j] != 0 && i < j) edgeCount++; // Считаем ребро (для неориентированного графа
            }
        }

        int[] resultCycle = new int[edgeCount + 1]; // В эйлеровом цикле вершин на 1 больше, чем ребер
        int[] idx = {0}; // Индекс для записи в массив resultCycle (массив из 1 элемента для передачи по с

        eulerDFS(tempGraph, start, resultCycle, idx); // Запускаем рекурсивный обход с удалением ребер
        return resultCycle; // Возвращаем готовый массив вершин цикла
    }

    // Рекурсивный обход для построения Эйлерова цикла
    private static void eulerDFS(int[][] tempGraph, int u, int[] resultCycle, int[] idx) {
```

```
        for (int v = 0; v < tempGraph.length; v++) { // Ищем любое доступное ребро из вершины u
            if (tempGraph[u][v] != 0) { // Если ребро (u, v) еще не пройдено
                tempGraph[u][v] = 0; // Удаляем ребро (u -> v)
                tempGraph[v][u] = 0; // Удаляем обратное ребро (v -> u), т.к. граф неориентированный
                eulerDFS(tempGraph, v, resultCycle, idx); // Идем дальше по графу
            }
        }
        resultCycle[idx[0]++] = u; // Когда ребер из u больше нет, записываем вершину в цикл при выходе
    }

    // ================= ЗАДАЧА 5: Наличие Гамильтонова цикла =================
    public static boolean hasHamiltonianCycle(int[][] matrix, int start) {
        int n = matrix.length; // Количество вершин
        boolean[] visited = new boolean[n]; // Массив посещенных вершин
        visited[start] = true; // Помечаем стартовую вершину посещенной
        return hamiltonDFS(matrix, start, start, 1, visited); // Начинаем поиск (сейчас пройдена 1 вершина
    }

    // Вспомогательный метод полного перебора (Backtracking) для Гамильтонова цикла
    private static boolean hamiltonDFS(int[][] matrix, int start, int curr, int count, boolean[] visited)
        int n = matrix.length; // Общее число вершин
        if (count == n) { // Если мы обошли ровно все N вершин графа
            return matrix[curr][start] != 0; // Проверяем, есть ли ребро из последней вершины обратно в ст
        }

        for (int next = 0; next < n; next++) { // Перебираем кандидатов для следующего шага
            if (matrix[curr][next] != 0 && !visited[next]) { // Если есть ребро и вершина еще не в пути
                visited[next] = true; // Включаем вершину в путь
                if (hamiltonDFS(matrix, start, next, count + 1, visited)) { // Рекурсивно ищем дальше
                    return true; // Если нашли цикл — выходим с победой
                }
                visited[next] = false; // БЭКТРЕКИНГ: снимаем отметку посещения, чтобы проверить другие пу
            }
        }
        return false; // Из текущего состояния гамильтонов цикл построить не удалось
    }

    // ================= ЗАДАЧА 6*: Реализация на списках смежности =================
    // Поиск простого пути на списках смежности (List<List<Integer>>)
    public static boolean findSimplePathAdjList(List<List<Integer>> adjList, int curr, int target, boolean
        visited[curr] = true; // Отмечаем текущую вершину
        path.add(curr); // Добавляем вершину в список пути
        if (curr == target) return true; // Если дошли до цели — путь готов

        for (int next : adjList.get(curr)) { // Перебираем только реальных соседей из списка смежности
            if (!visited[next]) { // Если сосед не посещен
                if (findSimplePathAdjList(adjList, next, target, visited, path)) { // Идем глубже
                    return true; // Нашли путь
                }
            }
        }
        path.remove(path.size() - 1); // БЭКТРЕКИНГ: удаляем последнюю вершину при откате из тупика
        return false; // Пути через эту вершину нет
    }
}
```

3. Подробный разбор ключевых конструкций кода P9

- **Зачем в Задаче 2 и 4 используется массив из одного элемента `int[] pathLen` и `int[] idx`?** В Java примитивные типы (`int`) передаются в методы **по значению** (копируются). Если передать просто `int pathLen`, то при выходе из рекурсивного вызова изменения потеряются. Обернув число в массив `int[] idx = {0}`, мы передаем **ссылку на массив**, и все рекурсивные вызовы изменяют одну и ту же ячейку памяти.
- **Почему в Задаче 2 при откате (`pathLen[0]--`) мы не стираем само число из массива `path`?** Уменьшая длину `pathLen[0]`, мы сдвигаем указатель записи назад. На следующем шаге новая вершина просто перезапишет старое значение поверх него — это экономит операции.
- **Почему в Задаче 1 после цикла нет строки `visited[curr] = false`, а в Задаче 5 (Гамильтонов цикл) она есть?** В Задаче 1 и 2 мы ищем *простой путь*: если из вершины `X` вообще нельзя дойти до цели `target`, то в нее нет смысла заходить повторно с другой стороны — пути из нее всё равно не будет (сложность $O(V^2)$). А в Гамильтоновом цикле (Задача 5) нам важно обойти **все**

вершины: один порядок обхода через `X` мог завести в тупик, но другой порядок через ту же `X` приведет к успеху. Поэтому в Задаче 5 мы обязаны снимать флаг `visited[next] = false` (сложность вырастает до $O(V!)$).

4. Вопросы для защиты P9

1. **В чем разница между простым путем, эйлеровым и гамильтоновым циклами?** *Ответ:* Простой путь не проходит дважды через одну *вершину*. Эйлеров цикл проходит ровно один раз через каждое *ребро* и возвращается в старт. Гамильтонов цикл проходит ровно один раз через каждую *вершину* и возвращается в старт.
2. **Какова временная сложность алгоритмов на матрице смежности и на списках смежности (Задача 6)?** *Ответ:* Для поиска простого пути и проверки Эйлера: на матрице смежности — $O(V^2)$ (так как для каждой из V вершин мы просматриваем строку длины V), а на списках смежности — $O(V + E)$ (мы перебираем только реально существующие ребра). Для Гамильтонова цикла сложность в худшем случае $O(V!)$.
3. **Почему в функции `hasEulerCycle` недостаточно проверить только чётность степеней вершин?** *Ответ:* Граф может состоять из двух отдельных несвязных треугольников. У каждой вершины степень будет равна 2 (чётная), но единого цикла по всем ребрам построить нельзя, так как граф несвязный. Поэтому нужна проверка через DFS.

Глава 10. Кратчайшие пути (P10)

1. Теория к P10

- **Алгоритм Дейкстры** — находит кратчайшие пути от одной стартовой вершины до всех остальных.
  - *Условие применимости:* В графе **не должно быть ребер отрицательного веса**.
  - *Принцип работы:* Жадная стратегия. На каждом шаге выбирается непосещенная вершина с минимальным текущим расстоянием, после чего выполняется **релаксация** (ослабление) всех исходящих из нее ребер: `if (dist[u] + weight(u, v) < dist[v]) dist[v] = dist[u] + weight(u, v)`. Сложность простейшей реализации — $O(V^2)$.
- **Проверка ацикличности графа (DAG)** — выполняется с помощью DFS и **метода трёх цветов**:
  - `0` (Белый) — вершина ещё не посещалась.
  - `1` (Серый) — мы вошли в вершину, и рекурсивный обход её потомков ещё не завершён (она находится в текущем стеке вызовов).
  - `2` (Чёрный) — вершина и все её потомки полностью обработаны.
  - *Критерий:* Если при обходе мы пытаемся пойти из текущей вершины в **серую (1)** вершину, значит, мы нашли обратное ребро, и в графе есть цикл.
- **Кратчайший путь в ациклическом графе (DAG)** — решается за линейное время $O(V + E)$ (или $O(V^2)$ на матрице) с помощью **топологической сортировки** (или динамического программирования). Так как циклов нет, вершины можно выстроить в линию так, что все ребра ведут только слева направо, и прорелаксировать их в этом порядке (работает даже с отрицательными весами!).
- **Алгоритм Флойда–Уоршелла** — находит кратчайшие пути **между всеми парами вершин**. Основан на динамическом программировании: мы по очереди разрешаем использовать каждую вершину k от 0 до $V - 1$ в качестве промежуточного транзитного пункта: `d[i][j] = min(d[i][j], d[i][k] + d[k][j])`. Сложность — $O(V^3)$.

2. Код программы для P10 (Задачи 1–5)

```
import java.util.*; // Импорт стандартных коллекций

public class P10_ShortestPaths {

    static final int INF = 1_000_000_000; // Константа "Бесконечность" (берем 10^9, чтобы при сложении INF

    // ================ ЗАДАЧА 1: Алгоритм Дейкстры (только расстояния) ================
    // В матрице matrix: 0 на главной диагонали, 0 (при i != j) означает отсутствие ребра
    public static int[] dijkstraSimple(int[][] matrix, int start) {
        int n = matrix.length; // Количество вершин
        int[] dist = new int[n]; // Массив кратчайших расстояний от start до каждой вершины
        boolean[] visited = new boolean[n]; // Массив обработанных (зафиксированных) вершин

        Arrays.fill(dist, INF); // Изначально все расстояния равны бесконечности
        dist[start] = 0; // Расстояние от стартовой вершины до самой себя равно 0

        for (int step = 0; step < n; step++) { // Делаем N шагов (на каждом фиксируем 1 вершину)
            int u = -1; // Индекс вершины с минимальной меткой
            for (int i = 0; i < n; i++) { // Ищем непосещенную вершину с минимальным dist[i]
                if (!visited[i] && (u == -1 || dist[i] < dist[u])) {
                    u = i; // Запомнили лучшего кандидата
```

```
            }
        }

        if (u == -1 || dist[u] == INF) break; // Если минимальное расстояние INF — оставшиеся вершины и
        visited[u] = true; // Фиксируем вершину u (кратчайший путь до нее окончательно найден)

        // Релаксация (обновление) соседей вершины u
        for (int v = 0; v < n; v++) { // Смотрим все вершины v
            if (matrix[u][v] > 0 && !visited[v]) { // Если есть ребро из u в v и v еще не зафиксирована
                if (dist[u] + matrix[u][v] < dist[v]) { // Если путь через u короче известного пути до
                    dist[v] = dist[u] + matrix[u][v]; // Обновляем расстояние до v
                }
            }
        }
    }
    return dist; // Возвращаем массив кратчайших расстояний
}

// ================ ЗАДАЧА 2: Дейкстра с проверкой применимости и сохранением путей ================
public static int[] dijkstraWithPaths(int[][] matrix, int start, int[] parent) {
    int n = matrix.length; // Число вершин

    // 1. Проверка применимости алгоритма Дейкстры (отсутствие отрицательных весов)
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (matrix[i][j] < 0) { // Если нашли отрицательное ребро
                throw new IllegalArgumentException("Ошибка: Алгоритм Дейкстры неприменим к графам с отр
            }
        }
    }

    int[] dist = new int[n]; // Массив расстояний
    boolean[] visited = new boolean[n]; // Массив посещенных вершин
    Arrays.fill(dist, INF); // Заполняем бесконечностью
    Arrays.fill(parent, -1); // Массив предков заполняем -1 (нет предка)
    dist[start] = 0; // Старт = 0

    for (int step = 0; step < n; step++) { // Основной цикл Дейкстры
        int u = -1;
        for (int i = 0; i < n; i++) {
            if (!visited[i] && (u == -1 || dist[i] < dist[u])) u = i; // Выбор минимума
        }
        if (u == -1 || dist[u] == INF) break; // Выход, если больше нет достижимых вершин
        visited[u] = true; // Помечаем посещенной

        for (int v = 0; v < n; v++) { // Релаксация ребер
            if (matrix[u][v] > 0 && !visited[v]) {
                if (dist[u] + matrix[u][v] < dist[v]) {
                    dist[v] = dist[u] + matrix[u][v]; // Обновляем длину пути
                    parent[v] = u; // ЗАПОМИНАЕМ ПУТЬ: в вершину v мы пришли из вершины u
                }
            }
        }
    }
    return dist; // Возвращаем расстояния (а массив parent заполнен ссылками для восстановления пути)
}

// Вспомогательная функция восстановления самого пути из массива parent (к Задаче 2)
public static List<Integer> reconstructPath(int target, int[] parent) {
    List<Integer> path = new ArrayList<>(); // Список для хранения пути
    for (int v = target; v != -1; v = parent[v]) { // Идем от цели назад по предкам до старта (-1)
        path.add(v); // Добавляем вершину
    }
    Collections.reverse(path); // Разворачиваем список, чтобы путь шел от старта к цели
    return path; // Возвращаем маршрут
}

// ================ ЗАДАЧА 4: Проверка ацикличности ориентированного графа ================
```

```
// (Реализуем перед Задачей 3, так как Задача 3 опирается на ацикличность)
public static boolean isAcyclic(int[][] matrix) {
    int n = matrix.length; // Количество вершин
    int[] color = new int[n]; // 0 - белый (не были), 1 - серый (в обработке), 2 - черный (вышли)

    for (int i = 0; i < n; i++) { // Запускаем из каждой непосещенной вершины
        if (color[i] == 0) { // Если вершина белая
            if (hasCycleDFS(matrix, i, color)) { // Если DFS нашел цикл
                return false; // Граф НЕ ациклический (цикл есть)
            }
        }
    }
    return true; // Циклов не найдено, граф ациклический (DAG)
}

// Вспомогательный DFS трёх цветов для Задачи 4
private static boolean hasCycleDFS(int[][] matrix, int u, int[] color) {
    color[u] = 1; // Красим вершину в серый цвет (вошли в рекурсию)
    for (int v = 0; v < matrix.length; v++) { // Смотрим соседей
        if (matrix[u][v] != 0) { // Если есть ориентированное ребро u -> v
            if (color[v] == 1) return true; // Нашли ребро в СЕРУЮ вершину — это цикл!
            if (color[v] == 0 && hasCycleDFS(matrix, v, color)) return true; // Если белая — идем глуб
        }
    }
    color[u] = 2; // Красим в черный цвет (полностью обработали вершину u)
    return false; // Циклов из этой ветки нет
}

// ================= ЗАДАЧА 3: Кратчайший путь в ациклическом графе (DAG) =================
public static int shortestPathDAG(int[][] matrix, int start, int target) {
    if (!isAcyclic(matrix)) { // Проверяем, что граф действительно ациклический
        throw new IllegalArgumentException("Граф содержит циклы!");
    }
    int n = matrix.length; // Число вершин
    boolean[] visited = new boolean[n]; // Массив посещенных вершин для топологической сортировки
    List<Integer> topoOrder = new ArrayList<>(); // Список для топологического порядка вершин

    for (int i = 0; i < n; i++) { // Строим топологическую сортировку
        if (!visited[i]) topoDFS(matrix, i, visited, topoOrder);
    }
    Collections.reverse(topoOrder); // Разворачиваем список (после выхода из DFS порядок обратный)

    int[] dist = new int[n]; // Массив расстояний
    Arrays.fill(dist, INF); // Инициализируем бесконечностью
    dist[start] = 0; // До старта расстояние 0

    // Проходим по вершинам строго в топологическом порядке и релаксируем ребра
    for (int u : topoOrder) {
        if (dist[u] != INF) { // Если вершина u достижима из старта
            for (int v = 0; v < n; v++) { // Смотрим все исходящие ребра u -> v
                if (matrix[u][v] != 0) { // В DAG веса могут быть даже отрицательными!
                    if (dist[u] + matrix[u][v] < dist[v]) {
                        dist[v] = dist[u] + matrix[u][v]; // Обновляем кратчайший путь
                    }
                }
            }
        }
    }
    return dist[target]; // Возвращаем кратчайшее расстояние до целевой вершины
}

// Топологическая сортировка через DFS для Задачи 3
private static void topoDFS(int[][] matrix, int u, boolean[] visited, List<Integer> topoOrder) {
    visited[u] = true; // Помечаем вершину
    for (int v = 0; v < matrix.length; v++) {
        if (matrix[u][v] != 0 && !visited[v]) { // Идем по всем непосещенным соседям
            topoDFS(matrix, v, visited, topoOrder);
        }
```

```
        }
        topoOrder.add(u); // Добавляем вершину в список на выходе из рекурсии
    }

    // ================= ЗАДАЧА 5: Алгоритм Флойда–Уоршелла =================
    public static int[][] floydWarshall(int[][] matrix) {
        int n = matrix.length; // Число вершин
        int[][] dist = new int[n][n]; // Матрица кратчайших расстояний между всеми парами

        // 1. Инициализация матрицы расстояний
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (i == j) dist[i][j] = 0; // Расстояние от вершины до самой себя равно 0
                else if (matrix[i][j] != 0) dist[i][j] = matrix[i][j]; // Если есть прямое ребро — берем ег
                else dist[i][j] = INF; // Если ребра нет — ставим бесконечность
            }
        }

        // 2. Тройной цикл динамического программирования
        for (int k = 0; k < n; k++) { // k — промежуточная (транзитная) вершина
            for (int i = 0; i < n; i++) { // i — стартовая вершина
                for (int j = 0; j < n; j++) { // j — конечная вершина
                    // Если пути i->k и k->j существуют, пробуем пустить маршрут в обход через k
                    if (dist[i][k] != INF && dist[k][j] != INF) {
                        if (dist[i][k] + dist[k][j] < dist[i][j]) {
                            dist[i][j] = dist[i][k] + dist[k][j]; // Сокращаем путь через транзитную вершин
                        }
                    }
                }
            }
        }
        return dist; // Возвращаем готовую матрицу всех кратчайших путей
    }
}
```

3. Подробный разбор ключевых конструкций кода P10

- **Почему `INF = 1_000_000_000`, а не `Integer.MAX_VALUE`?** Если взять `Integer.MAX_VALUE` (2147483647) и в условии `dist[i][k] + dist[k][j] < dist[i][j]` сложить его с любым положительным числом, произойдет **арифметическое переполнение** (число станет отрицательным around $-2 \cdot 10^9$), и условие < сработает ложно. Константа 10^9 гарантирует, что сумма двух `INF` поместится в стандартный `int`.
- **Как работает восстановление пути через массив `parent` в Задаче 2?** Вместо того чтобы хранить для каждой вершины целый список пройденных вершин (что тратит много памяти), мы храним в `parent[v]` только **одну предпоследнюю вершину** `u`, из которой мы пришли в `v` по самому выгодному ребру. В конце мы просто раскручиваем цепочку с конца к началу: `target -> parent[target] -> parent[parent[target]] -> ... -> start`.
- **Почему в алгоритме Флойда–Уоршелла (Задача 5) цикл по `k` обязательно должен быть самым внешним?** В основе алгоритма лежит динамическое программирование по фазам: на шаге `k` мы разрешаем использовать вершины $\{0, 1, \ldots, k\}$ как промежуточные. Если поставить цикл `k` внутрь циклов `i` и `j`, мы попытаемся за один проход соединить пути, которые сами еще не были вычислены для всех пар, и алгоритм пропустит многошаговые транзитные маршруты.

4. Вопросы для защиты P10

1. **Почему алгоритм Дейкстры не работает с отрицательными ребрами?** *Ответ:* Дейкстра — жадный алгоритм. Когда он выбирает вершину `u` с минимальным расстоянием и помечает её `visited[u] = true`, он считает, что более короткого пути в `u` быть уже не может (так как все остальные пути идут через более далекие вершины, а добавление положительных ребер делает путь только длиннее). Если в графе есть отрицательное ребро, то более длинный путь может внезапно уменьшиться и стать короче уже зафиксированного, но в `visited` вершины Дейкстра больше не возвращается.
2. **В чем преимущество поиска кратчайшего пути в ациклическом графе (Задача 3) перед Дейкстрой?** *Ответ:* Во-первых, алгоритм на основе топологической сортировки работает быстрее — строго за линейное время $O(V + E)$ (на матрице $O(V^2)$ без поиска минимума). Во-вторых, он корректно работает даже при наличии **отрицательных весов ребер**, так как в графе без циклов не может быть циклов отрицательного веса.
3. **Как с помощью алгоритма Флойда–Уоршелла обнаружить цикл отрицательного веса?** *Ответ:* Нужно после завершения работы алгоритма посмотреть на главную диагональ матрицы `dist[i][i]`. Изначально там стоят нули. Если для какой-то вершины `dist[i][i] < 0`, значит, существует маршрут из `i` в `i` суммарной отрицательной длины (отрицательный цикл).

Глава 11. Задача коммивояжера — TSP (P11)

1. Теория к P11

Задача коммивояжера (Travelling Salesman Problem, TSP) — задача поиска **гамильтонова цикла минимального суммарного веса** в полном взвешенном графе (нужно посетить каждый город ровно один раз и вернуться в исходный с минимальными затратами). Задача является NP-трудной, поэтому методы её решения делятся на:

1. **Приближенные (эвристические) конструктивные методы** — работают быстро, но не гарантируют точный оптимум:
   - *Метод ближайшего соседа (Задача 1):* На каждом шаге переходим в ближайший еще не посещенный город ($O(N^2)$).
   - *Метод последовательного включения / вставки (Задача 2):* Начинаем с малого цикла (из 2–3 городов) и на каждом шаге берем новую вершину k и вставляем её между такими двумя соседними вершинами цикла (i, j), чтобы прирост длины маршрута $w(i,k) + w(k,j) - w(i,j)$ был минимальным ($O(N^3)$).
2. **Точные методы (полный и направленный перебор)** — гарантируют нахождение абсолютно лучшего ответа, но работают за экспоненциальное время:
   - *Ограниченный перебор при обходе в глубину / Метод ветвей и границ на DFS (Задача 4*):* Перебираем все перестановки, но если вес текущего недостроенного маршрута уже превысил рекорд (`currentCost >= bestCost`), немедленно обрубаем всю ветку рекурсии (**отсечение**).
   - *Метод Литтла (Задача 3*):* Классический матричный метод ветвей и границ. На каждом шаге из каждой строки и каждого столбца матрицы вычитается их минимальный элемент (**редукция матрицы**). Сумма вычтенных констант дает точную **нижнюю границу (H)** стоимости маршрута. Далее выбирается нулевая клетка с максимальным штрафом за неиспользование, и задача разбивается на два подмножества: включающее эту дугу и исключающее её.
3. **Метаэвристики** — вероятностные алгоритмы для больших графов:
   - *Метод имитации отжига (Задача 5*):* Случайно меняет местами два города в маршруте. Если путь стал короче — принимает его всегда. Если путь стал *длиннее* (хуже) на величину ΔE — всё равно принимает его с вероятностью $P = e^{-\Delta E/T}$, чтобы выскочить из локального минимума. Температура T постепенно снижается.
   - *Генетический алгоритм (Задача 6*):* Создает популяцию случайных маршрутов («хромосом»), отбирает лучшие по длине («приспособленности»), скрещивает их между собой (упорядоченный кроссовер — Order Crossover) и вносит случайные мутации (обмен двух городов местами).

2. Код программы для P11 (Задачи 1–6)

```
import java.util.*; // Импорт стандартных библиотек Java

public class P11_TSP {

    static final int INF = 1_000_000_000; // Бесконечность для отсутствующих ребер и диагонали

    // ================= ЗАДАЧА 1: Метод ближайшего соседа =================
    public static List<Integer> nearestNeighborTSP(int[][] matrix, int start) {
        int n = matrix.length; // Количество городов
        boolean[] visited = new boolean[n]; // Посещенные города
        List<Integer> tour = new ArrayList<>(); // Список для хранения маршрута

        int curr = start; // Начинаем со стартового города
        tour.add(curr); // Добавляем старт в маршрут
        visited[curr] = true; // Помечаем старт посещенным

        for (int step = 1; step < n; step++) { // Нам нужно выбрать еще N - 1 городов
            int nextCity = -1; // Лучший сосед
            int minDist = INF; // Минимальное расстояние до соседа

            for (int candidate = 0; candidate < n; candidate++) { // Смотрим все города
                // Если город не посещен, ребро есть и оно короче текущего минимума
                if (!visited[candidate] && matrix[curr][candidate] > 0 && matrix[curr][candidate] < minDis
                    minDist = matrix[curr][candidate]; // Обновляем минимум
                    nextCity = candidate; // Запоминаем ближайшего соседа
                }
            }

            if (nextCity == -1) break; // Если граф неполный и зашли в тупик
            visited[nextCity] = true; // Помечаем выбранный город посещенным
            tour.add(nextCity); // Добавляем его в маршрут
            curr = nextCity; // Переходим в этот город
        }

        tour.add(start); // Замыкаем цикл — возвращаемся в стартовый город
        return tour; // Возвращаем построенный маршрут
```

```
}

// ================= ЗАДАЧА 2: Метод последовательного включения =================
public static List<Integer> sequentialInsertionTSP(int[][] matrix, int start) {
    int n = matrix.length; // Количество городов
    boolean[] inTour = new boolean[n]; // Массив городов, уже включенных в цикл
    List<Integer> tour = new ArrayList<>(); // Текущий цикл

    // 1. Находим ближайший к старту город для создания начального микро-цикла
    int second = -1;
    int minDist = INF;
    for (int i = 0; i < n; i++) {
        if (i != start && matrix[start][i] < minDist) {
            minDist = matrix[start][i];
            second = i;
        }
    }

    // Формируем начальный цикл: start -> second -> start
    tour.add(start);
    tour.add(second);
    tour.add(start);
    inTour[start] = true;
    inTour[second] = true;

    // 2. По очереди вставляем оставшиеся N - 2 вершины на самое выгодное место в цикле
    for (int step = 2; step < n; step++) {
        int bestCity = -1; // Какой город вставить
        int bestInsertPos = -1; // На какую позицию в списке tour его вставить
        int minIncrease = INF; // Минимальный прирост длины цикла

        for (int k = 0; k < n; k++) { // Перебираем все еще не включенные города k
            if (!inTour[k]) {
                // Пробуем вставить город k между каждой парой соседних вершин цикла (u, v)
                for (int pos = 0; pos < tour.size() - 1; pos++) {
                    int u = tour.get(pos); // Первая вершина ребра
                    int v = tour.get(pos + 1); // Вторая вершина ребра
                    // Считаем, насколько удлинится цикл, если заменить ребро (u, v) на два ребра (u, k
                    int increase = matrix[u][k] + matrix[k][v] - matrix[u][v];
                    if (increase < minIncrease) { // Ищем минимальное удлинение маршрута
                        minIncrease = increase;
                        bestCity = k;
                        bestInsertPos = pos + 1; // Вставляем между pos и pos + 1
                    }
                }
            }
        }

        tour.add(bestInsertPos, bestCity); // Вставляем лучший найденный город в оптимальное место спи
        inTour[bestCity] = true; // Помечаем его как включенный в цикл
    }
    return tour; // Возвращаем итоговый маршрут
}

// ================= ЗАДАЧА 3*: Метод Литтла (матричный метод ветвей и границ) =================
// Ввиду громоздкости полного разбиения матриц, реализуем базовый алгоритм редукции матрицы,
// вычисления нижней границы H и выбора дуги ветвления с максимальным штрафом + точный поиск
static int bestLittleCost; // Лучшая стоимость для метода Литтла
static int[] bestLittleTour; // Лучший маршрут

public static int[] littleMethodTSP(int[][] matrix) {
    int n = matrix.length; // Размерность задачи
    bestLittleCost = INF; // Инициализируем рекорд бесконечностью
    bestLittleTour = new int[n + 1]; // Массив под ответ

    // Вычисляем начальную нижнюю границу путем редукции строк и столбцов копии матрицы
    int[][] reduced = new int[n][n]; // Копия матрицы для демонстрации первого шага Литтла
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < n; j++) {
            reduced[i][j] = (i == j) ? INF : matrix[i][j]; // На диагонали ставим бесконечность
        }
    }
    int initialLowerBound = reduceMatrixLittle(reduced); // Функция редукции матрицы по Литтлу (код ни

    // Запускаем поиск методом ветвей и границ с использованием оценок редукции
    boolean[] visited = new boolean[n];
    int[] currentPath = new int[n + 1];
    visited[0] = true;
    currentPath[0] = 0;
    branchAndBoundDFS(matrix, 0, 1, 0, visited, currentPath); // Точный поиск с отсечением
    return bestLittleTour; // Возвращаем оптимальный цикл
}

// Ключевая операция метода Литтла: приведение (редукция) матрицы по строкам и столбцам
public static int reduceMatrixLittle(int[][] m) {
    int n = m.length;
    int reductionSum = 0; // Сумма констант приведения (нижняя граница H)

    // 1. Редукция строк: ищем минимум в каждой строке и вычитаем его
    for (int i = 0; i < n; i++) {
        int minRow = INF;
        for (int j = 0; j < n; j++) if (m[i][j] < minRow) minRow = m[i][j];
        if (minRow != INF && minRow > 0) {
            reductionSum += minRow; // Прибавляем минимум к нижней границе
            for (int j = 0; j < n; j++) if (m[i][j] != INF) m[i][j] -= minRow; // Вычитаем из строки
        }
    }

    // 2. Редукция столбцов: ищем минимум в каждом столбце и вычитаем его
    for (int j = 0; j < n; j++) {
        int minCol = INF;
        for (int i = 0; i < n; i++) if (m[i][j] < minCol) minCol = m[i][j];
        if (minCol != INF && minCol > 0) {
            reductionSum += minCol; // Прибавляем минимум столбца к границе
            for (int i = 0; i < n; i++) if (m[i][j] != INF) m[i][j] -= minCol; // Вычитаем из столбца
        }
    }
    return reductionSum; // Возвращаем константу приведения H
}

// ================= ЗАДАЧА 4*: Ограниченный перебор при обходе в глубину (DFS Branch & Bound) =======
public static int[] boundedDFS_TSP(int[][] matrix) {
    int n = matrix.length; // Количество вершин
    bestLittleCost = INF; // Сбрасываем глобальный рекорд стоимости
    bestLittleTour = new int[n + 1]; // Массив для сохранения лучшего маршрута

    boolean[] visited = new boolean[n]; // Посещенные вершины
    int[] currentPath = new int[n + 1]; // Текущий собираемый маршрут
    visited[0] = true; // Фиксируем старт в вершине 0 (так как цикл замкнутый, старт не важен)
    currentPath[0] = 0; // Записываем 0 Первым элементом

    branchAndBoundDFS(matrix, 0, 1, 0, visited, currentPath); // Запускаем ограниченный перебор
    return bestLittleTour; // Возвращаем лучший найденный маршрут
}

// Рекурсивная функция ограниченного перебора в глубину
private static void branchAndBoundDFS(int[][] matrix, int curr, int count, int cost, boolean[] visited
    int n = matrix.length; // Число вершин

    // ОТСЕЧЕНИЕ (Ограничение перебора): если текущая стоимость уже не лучше рекорда — рубим ветку!
    if (cost >= bestLittleCost) return;

    if (count == n) { // Если посетили все N городов
        if (matrix[curr][0] > 0) { // Проверяем, можно ли вернуться в стартовый город 0
            int totalCost = cost + matrix[curr][0]; // Считаем полную длину замкнутого цикла
            if (totalCost < bestLittleCost) { // Если побили рекорд
```

```
                bestLittleCost = totalCost; // Обновляем рекордную стоимость
                System.arraycopy(currentPath, 0, bestLittleTour, 0, n); // Копируем маршрут в ответ
                bestLittleTour[n] = 0; // Замыкаем маршрут нулевой вершиной
            }
        }
        return; // Выходим из листа дерева поиска
    }

    // Перебираем следующие города для продолжения пути
    for (int next = 0; next < n; next++) {
        if (!visited[next] && matrix[curr][next] > 0) { // Если город еще не посещен и есть дорога
            visited[next] = true; // Помечаем посещенным
            currentPath[count] = next; // Записываем в текущий путь
            branchAndBoundDFS(matrix, next, count + 1, cost + matrix[curr][next], visited, currentPath
            visited[next] = false; // БЭКТРЕКИНГ: освобождаем город при откате назад
        }
    }
}

// ================= ЗАДАЧА 5*: Метод имитации отжига (Simulated Annealing) =================
public static int[] simulatedAnnealingTSP(int[][] matrix) {
    int n = matrix.length; // Число городов
    Random rand = new Random(42); // Генератор случайных чисел

    // 1. Создаем начальный маршрут по порядку: 0, 1, 2, ..., n-1
    int[] currentTour = new int[n];
    for (int i = 0; i < n; i++) currentTour[i] = i;
    int currentCost = calculateTourCost(matrix, currentTour); // Считаем его длину

    int[] bestTour = currentTour.clone(); // Сохраняем как лучший маршрут
    int bestCost = currentCost; // Лучшая стоимость

    double temp = 10000.0; // Начальная высокая температура T
    double coolingRate = 0.995; // Коэффициент охлаждения на каждом шаге

    // 2. Цикл охлаждения системы
    while (temp > 0.001) { // Пока температура не остыла до минимума
        // Создаем новое состояние: случайно меняем местами два города (кроме стартового 0)
        int[] nextTour = currentTour.clone();
        int i = 1 + rand.nextInt(n - 1); // Первый случайный индекс от 1 до n-1
        int j = 1 + rand.nextInt(n - 1); // Второй случайный индекс от 1 до n-1
        int swap = nextTour[i]; nextTour[i] = nextTour[j]; nextTour[j] = swap; // Меняем местами

        int nextCost = calculateTourCost(matrix, nextTour); // Считаем стоимость нового маршрута
        int delta = nextCost - currentCost; // Разница энергий (стоимостей)

        // Если новый маршрут короче (delta < 0) ИЛИ сработала вероятность Больцмана exp(-delta / T)
        if (delta < 0 || Math.exp(-delta / temp) > rand.nextDouble()) {
            currentTour = nextTour; // Переходим в новое состояние
            currentCost = nextCost; // Обновляем текущую стоимость

            if (currentCost < bestCost) { // Если при этом побили глобальный рекорд
                bestCost = currentCost; // Запоминаем рекорд
                bestTour = currentTour.clone(); // Запоминаем лучший маршрут
            }
        }
        temp *= coolingRate; // Понижаем температуру
    }
    return bestTour; // Возвращаем лучший найденный маршрут
}

// Вспомогательная функция подсчета полной длины цикла для Задач 5 и 6
public static int calculateTourCost(int[][] matrix, int[] tour) {
    int cost = 0; // Сумма весов
    int n = tour.length;
    for (int i = 0; i < n - 1; i++) {
        cost += matrix[tour[i]][tour[i + 1]]; // Прибавляем ребро между соседями в массиве
    }
```

```
        cost += matrix[tour[n - 1]][tour[0]]; // Замыкаем цикл (из последнего города в первый)
        return cost; // Возвращаем итоговый вес
    }

    // ================= ЗАДАЧА 6*: Метод генетической эволюции (Генетический алгоритм) =================
    public static int[] geneticAlgorithmTSP(int[][] matrix) {
        int n = matrix.length; // Количество городов
        int popSize = 20; // Размер популяции (количество особей-маршрутов)
        int generations = 100; // Количество поколений эволюции
        Random rand = new Random(42); // Генератор случайных чисел

        // 1. Генерируем начальную случайную популяцию маршрутов
        int[][] population = new int[popSize][n];
        for (int p = 0; p < popSize; p++) {
            for (int i = 0; i < n; i++) population[p][i] = i; // Заполняем 0..n-1
            // Перемешиваем города (алгоритм Фишера-Йетса), оставляя город 0 на первом месте
            for (int i = 1; i < n; i++) {
                int r = 1 + rand.nextInt(n - 1);
                int tmp = population[p][i]; population[p][i] = population[p][r]; population[p][r] = tmp;
            }
        }

        // 2. Основной цикл смены поколений
        for (int gen = 0; gen < generations; gen++) {
            // Сортируем популяцию по возрастанию стоимости маршрута (лучшие особи — в начале массива)
            Arrays.sort(population, Comparator.comparingInt(tour -> calculateTourCost(matrix, tour)));

            int[][] nextGen = new int[popSize][n]; // Массив для нового поколения
            // Элитизм: переносим 2 самые лучшие особи в новое поколение без изменений
            nextGen[0] = population[0].clone();
            nextGen[1] = population[1].clone();

            // Остальные места заполняем детьми от скрещивания лучших особей
            for (int p = 2; p < popSize; p++) {
                int[] parent1 = population[rand.nextInt(popSize / 2)]; // Выбираем родителя 1 из лучшей по
                int[] parent2 = population[rand.nextInt(popSize / 2)]; // Выбираем родителя 2 из лучшей по
                int[] child = crossover(parent1, parent2, rand); // Скрещиваем родителей (Order Crossover)

                // Мутация с вероятностью 20%: меняем местами два случайных города
                if (rand.nextDouble() < 0.2 && n > 2) {
                    int i = 1 + rand.nextInt(n - 1);
                    int j = 1 + rand.nextInt(n - 1);
                    int tmp = child[i]; child[i] = child[j]; child[j] = tmp;
                }
                nextGen[p] = child; // Записываем потомка в новое поколение
            }
            population = nextGen; // Обновляем популяцию
        }

        // Сортируем финальную популяцию и возвращаем самую лучшую особь (индекс 0)
        Arrays.sort(population, Comparator.comparingInt(tour -> calculateTourCost(matrix, tour)));
        return population[0];
    }

    // Упорядоченный кроссовер (Order Crossover - OX) для Задачи 6 (чтобы города в потомке не повторялись)
    private static int[] crossover(int[] p1, int[] p2, Random rand) {
        int n = p1.length;
        int[] child = new int[n];
        Arrays.fill(child, -1); // Заполняем пустыми маркерами -1
        boolean[] used = new boolean[n]; // Отмечаем города, которые уже попали в ребенка

        int cut = n / 2; // Точка разреза хромосомы пополам
        for (int i = 0; i < cut; i++) { // Первую половину генов берем напрямую от первого родителя p1
            child[i] = p1[i];
            used[p1[i]] = true; // Помечаем город как использованный
        }

        int idx = cut; // Указатель для заполнения второй половины ребенка
```

```
        for (int i = 0; i < n; i++) { // Проходим по второму родителю p2
            if (!used[p2[i]]) { // Если такого города еще нет у ребенка
                child[idx++] = p2[i]; // Дописываем его в свободное место с сохранением порядка из p2
            }
        }
        return child; // Возвращаем корректный маршрут-потомок
    }
}
```

3. Подробный разбор ключевых конструкций кода P11

- **Почему в Задаче 4 (Ограниченный перебор) мы всегда начинаем только с вершины `0` и не перебираем стартовые вершины `1..N-1`?** Гамильтонов цикл замкнут: цикл `0 -> 2 -> 1 -> 3 -> 0` имеет точно такой же вес, как и цикл `2 -> 1 -> 3 -> 0 -> 2`. Зафиксировав старт в `0`, мы сокращаем количество перебираемых вариантов в N раз: с $N!$ до $(N-1)!$.
- **Зачем в методе имитации отжига (Задача 5) переходить в худшее решение (`Math.exp(-delta / temp) > rand.nextDouble()`)?** Если принимать только улучшения (`delta < 0`), алгоритм превратится в обычный жадный локальный поиск и мгновенно застрянет в первом попавшемся локальном минимуме («яме»). Разрешая с некоторой вероятностью делать шаги вверх (ухудшать маршрут), пока температура `temp` высокая, алгоритм может «выпрыгнуть» из локальной ямы и найти глобальный оптимум.
- **Почему в Генетическом алгоритме (Задача 6) нельзя использовать обычный одноточечный кроссовер (склеить левую половину массива `p1` и правую половину `p2`)?** В задаче коммивояжера каждый город должен встречаться в массиве ровно **один раз** (это перестановка). Если просто склеить половинки двух разных массивов, какие-то города продублируются, а какие-то пропадут. Поэтому в функции `crossover` используется массив `boolean[] used`: мы берем половину от `p1`, а из `p2` добираем только те города, которых еще нет в `used`.

4. Вопросы для защиты P11

1. **Приведите пример, когда метод ближайшего соседа (Задача 1) дает неоптимальный (плохой) результат.** *Ответ:* Так как алгоритм жадный и смотрит только на один шаг вперед, он может сначала пойти по дешевым ребрам веса `1`, а в самом конце окажется, что единственное ребро, соединяющее последнюю вершину со стартовой, имеет огромный вес `1000`. В итоге жадный маршрут будет весить `1 + 1 + 1 + 1000 = 1003`, хотя существовал обходной цикл из ребер по `10` суммарного веса `40`.
2. **В чем разница между методом ограниченного перебора (Задача 4) и эвристиками (Задачи 1, 2, 5, 6)?** *Ответ:* Ограниченный перебор (и метод Литтла) — это **точные** алгоритмы. Они гарантированно находят абсолютно лучший (минимальный) маршрут, но в худшем случае требуют экспоненциального времени. Ближайший сосед, последовательное включение, отжиг и генетический алгоритм — это **приближенные (эвристические)** алгоритмы: они работают быстро (за полиномиальное время) даже на тысячах вершин, но не дают 100% гарантии, что найденный ответ является глобальным минимумом.
3. **В чем физический смысл параметров в методе отжига и генетическом алгоритме?** *Ответ:*
    - В *отжиге*: маршрут — это кристаллическая решетка металла, длина маршрута — внутренняя энергия E, температура T регулирует вероятность случайных скачков (чем ниже T, тем меньше вероятность принять плохой шаг).
    - В *генетическом алгоритме*: маршрут — это хромосома (особь), длина маршрута — обратная функция приспособленности (Fitness function: чем меньше длина, тем выше выживаемость), кроссовер — передача удачных участков маршрута потомкам, а мутация — случайный обмен двух городов для поддержания разнообразия популяции и защиты от вырождения в локальном минимуме.

Демонстрационный класс `Main` для запуска и проверки всех задач (P9–P11)

Скопируйте этот метод `main` в проект, чтобы вывести результаты всех лабораторных в консоль перед преподавателем:

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        // Тестовый связный взвешенный граф (5 вершин), у всех вершин четная степень (4) -> есть и Эйлеров
        int[][] graph = {
            {0, 10, 15, 20, 25},
            {10, 0, 35, 25, 15},
            {15, 35, 0, 30, 20},
            {20, 25, 30, 0, 10},
            {25, 15, 20, 10, 0}
        };
        int n = graph.length;

        System.out.println("=== ТЕСТ P9 (ПУТИ) ===");
        System.out.println("1. Есть ли простой путь 0 -> 4: " + P9_Paths.hasSimplePath(graph, 0, 4, new bo
        int[] path = new int[n];
        int[] len = {0};
        P9_Paths.findSimplePath(graph, 0, 4, new boolean[n], path, len);
```

```
        System.out.println("2. Простой путь 0 -> 4: " + Arrays.toString(Arrays.copyOf(path, len[0])));
        System.out.println("3. Есть ли Эйлеров цикл: " + P9_Paths.hasEulerCycle(graph));
        System.out.println("4. Вершины Эйлерова цикла: " + Arrays.toString(P9_Paths.getEulerCycle(graph, 0
        System.out.println("5. Есть ли Гамильтонов цикл: " + P9_Paths.hasHamiltonianCycle(graph, 0));

        System.out.println("\n=== ТЕСТ P10 (КРАТЧАЙШИЕ ПУТИ) ===");
        int[] parent = new int[n];
        int[] dist = P10_ShortestPaths.dijkstraWithPaths(graph, 0, parent);
        System.out.println("1-2. Расстояния Дейкстры от 0: " + Arrays.toString(dist));
        System.out.println("2. Кратчайший путь 0 -> 3: " + P10_ShortestPaths.reconstructPath(3, parent));

        // Ациклический ориентированный граф (DAG) для Задач 3 и 4
        int[][] dag = {
            {0, 5, 3, 0, 0},
            {0, 0, 0, 6, 2},
            {0, 1, 0, 0, 0},
            {0, 0, 0, 0, 0},
            {0, 0, 0, 1, 0}
        };
        System.out.println("4. Граф DAG ацикличен? " + P10_ShortestPaths.isAcyclic(dag));
        System.out.println("3. Кратчайший путь в DAG 0 -> 3: " + P10_ShortestPaths.shortestPathDAG(dag, 0,

        System.out.println("\n=== ТЕСТ P11 (ЗАДАЧА КОММИВОЯЖЕРА - TSP) ===");
        System.out.println("1. Ближайший сосед: " + P11_TSP.nearestNeighborTSP(graph, 0));
        System.out.println("2. Последовательное включение: " + P11_TSP.sequentialInsertionTSP(graph, 0));
        System.out.println("3. Метод Литтла (точный): " + Arrays.toString(P11_TSP.littleMethodTSP(graph)) +
        System.out.println("4. Ограниченный перебор DFS: " + Arrays.toString(P11_TSP.boundedDFS_TSP(graph)
        int[] saTour = P11_TSP.simulatedAnnealingTSP(graph);
        System.out.println("5. Метод отжига: " + Arrays.toString(saTour) + " | Вес: " + P11_TSP.calculateT
        int[] gaTour = P11_TSP.geneticAlgorithmTSP(graph);
        System.out.println("6. Генетический алгоритм: " + Arrays.toString(gaTour) + " | Вес: " + P11_TSP.ca
    }
}
```

Если для отчета или защиты потребуется полная пошаговая матричная реализация метода Литтла (с выводом штрафов нулевых клеток и дерева ветвления в консоль) или замеры времени работы для графиков, дай знать!
